Lecture 1 – Web Engineering

1. Recap – Basic Web Concepts

- **WWW (World Wide Web):** Internet-based service.
- **W3C:** World Wide Web Consortium, founded in 1994 by Tim Berners-Lee.
- **Web Browser:** Software to access the web.
- **Web Server:** Computer system that processes requests via HTTP.

2. Introduction to Web Engineering

What is Engineering?

- Practical application of science to commerce or industry. Goal: Design applications better – faster, cheaper, more secure, etc.

What is Software Engineering?

- Application of science and mathematics to make computer equipment useful via:
  - Computer programs
  - Procedures
  - Documentation

What is Web Engineering?

- An independent branch of software engineering.
- Systematic and quantifiable approaches for:
  - Specification
  - Implementation
  - Operation
  - Maintenance
- Ensures high-quality web applications.
- Scientific discipline studying these approaches.

3. Web Application Definition

- Software system based on **W3C technologies & standards**.
- Provides web-specific resources (content & services) via a web browser.
- Considered **full-fledged, complex software systems**.
- Requires **methodologically sound engineering approach**.

4. Basic Principles of Web Engineering

1. Clearly defined goals and requirements.
2. Systematic development in phases.
3. Careful planning of phases.
4. Continuous audit of the development process.

- Similar principles to Software Engineering.

5. Categories of Web Applications

Category	Description	Examples	Pros ✅	Cons ❌
Document-Centric Websites	Static HTML pages stored on a server, manually updated.	Brochure sites, personal homepages, info pages.	- Simple to develop - Stable - Fast load times - Low technical complexity	- High maintenance cost for frequent updates - Risk of outdated/inconsistent info - Manual updates required - Limited interactivity
Interactive Web Apps	Dynamic pages generated based on user input via forms, buttons, menus (CGI/HTML forms).	News sites, virtual exhibitions, timetables.	- Basic user interactivity - Dynamic content generation - Better user engagement than static sites	- Still limited functionality - Requires server-side processing - Can be slow with heavy traffic
Transactional Web Apps	Allow users to read **and update** content; rely on databases for consistency.	Online banking, e-commerce, booking systems.	- Supports user transactions - Real-time data updates - Enables e-commerce & business logic	- Complex to develop - Requires secure transactions - High database dependency - Needs robust error handling

Category	Description	Examples	Pros ✅	Cons ❌
Workflow-Based Web Apps	Support automated workflows within/between organizations; use Web services for interoperability.	B2B systems, e-government, healthcare workflows.	- Streamlines business processes - Supports cross-organization integration - Improves operational efficiency	- High complexity in workflow design - Requires robust error & exception handling - Integration challenges with legacy systems
Collaborative Web Apps	Enable multiple users to work together in real-time or asynchronously (groupware).	Wikis, e-learning platforms, scheduling tools, chat rooms.	- Enhances teamwork & communication - Supports shared workspaces - Flexible for unstructured tasks	- Can suffer from version conflicts - Requires moderation in open environments - May have usability challenges
Social Web Apps	Identity-driven platforms for social interaction within communities.	Blogs, social networks (Instagram, Facebook), forums.	- Builds communities & engagement - Encourages user-generated content - High user retention potential	- Privacy & security concerns - Moderation challenges - Can suffer from misinformation/spam
Portal-Oriented Web Apps	Single point of access to multiple services/information sources.	Yahoo, Google, business portals, community portals.	- Centralized access - Can integrate diverse services - Good for personalized dashboards	- Can become cluttered - Integration complexity - Requires constant updating

Category	Description	Examples	Pros ✅	Cons ❌
Ubiquitous Web Apps	Provide context-aware, personalized services anytime, anywhere, on any device.	Mobile restaurant menus, location-based services, IoT apps.	- Highly accessible & convenient - Personalized user experience - Supports mobile & multi-device use	- Must adapt to device limitations (screen, bandwidth) - High development complexity - Privacy concerns with location data
Semantic Web Apps	Present information in machine-readable form to enable knowledge linking, reuse, and automation.	Knowledge graphs, recommender systems, smart agents.	- Enables intelligent data processing - Improves search & recommendation accuracy - Supports automation & AI integration	- Still emerging technology - Requires standardized ontologies - High implementation complexity
Rich Internet Applications (RIAs)	Web apps with desktop-like features, running in browsers without installation.	Gmail, Google Docs, Trello, Figma.	- Rich user experience - No installation required - Responsive & interactive	- Can be heavy on resources - Browser compatibility issues - Security vulnerabilities (e.g., XSS)

6. Characteristics of Web Applications

1. Introduction to Web Application Characteristics

- Web applications differ from traditional software in **unique features**.
- Characteristics include:
  - **Absent in traditional apps** (e.g., non-linear navigation)
  - **More critical in Web apps** (e.g., update frequency)

- **Varies by application type**: e.g., transactional apps require higher content consistency than informational ones.
- Characteristics organized into **3 dimensions** (ISO/IEC 9126-1 standard):
    1. **Product**
    2. **Usage**
    3. **Development**
- **Evolution** is a **cross-cutting dimension** influencing all three.

2. Product-Related Characteristics

- **Three core components** of a Web application:
    1. **Content**
    2. **Hypertext structure**
    3. **Presentation (UI)**

2.1 Content

- **"Content is King"** – primary reason for using Web apps.
- **Roles**: Developers also act as **authors**.
- **Key aspects**:
    - **Document-centric & multimedia**: Mix of text, tables, images, audio, video.
    - **Dynamic content generation** (e.g., real-time updates).
    - **Quality demands**:
        - **Topicality** (e.g., news sites)
        - **Accuracy** (e.g., pricing in e-commerce)
        - **Consistency & reliability**
- **Challenge**: Ensuring quality despite **high volume and frequent updates**.

2.2 Hypertext

- **Non-linearity**: Core feature – users navigate freely.
- **Basic elements**:
    - **Nodes** (e.g., HTML documents)
    - **Links** (unidirectional on Web)
    - **Anchors** (source/destination of links)

- **Potential problems**:
    - **Disorientation**: Getting lost in non-linear space.
    - **Cognitive overload**: Managing multiple paths/tasks.
- **Solutions**:
    - Sitemaps, search, history, access logs.
    - Meaningful link naming, design patterns.

2.3 Presentation (User Interface)

- **Two critical features**:
    1. **Aesthetics** ("look & feel") – highly competitive, trend-driven.
    2. **Self-explanation** – no documentation needed; intuitive navigation.

3. Usage-Related Characteristics

- **High heterogeneity** in users, devices, access contexts.
- **Three context groups**:

3.1 Social Context (Users)

- **Spontaneity**: Users can leave anytime; **no loyalty**.
    - **Requires scalability** to handle unpredictable user numbers.
- **Multiculturality**: Diverse user groups in terms of:
    - Abilities (e.g., disabilities)
    - Knowledge (expertise)
    - Preferences (interests)
- **Personalization** needed to adapt content, hypertext, presentation.

3.2 Technical Context (Network & Devices)

- **Quality of Service (QoS)**:
    - Must adapt to **bandwidth, reliability, connection stability**.
    - Especially critical for **mobile and multimedia content**.
- **Multi-platform delivery**:
    - Support for **various devices** (different screens, memory, software).
    - **Browser diversity**: Different versions, configurations, plugins.

    - **Adaptation strategies**: Web clipping, printer versions, platform-independent libraries.

3.3 Natural Context (Location & Time)

- **Globality**:
    - **Internationalization**: Regional, cultural, linguistic adaptation.
    - **Location-aware services**: Use physical/logical location data.
    - **Security challenges**: Prevent unauthorized access to private areas.
- **Availability**:
    - **24/7 access** demands high stability.
    - **Instant delivery** requires high initial quality.
    - **Time-aware services** (e.g., timetables).

4. Development-Related Characteristics

4.1 Development Team

- **Multidisciplinary**: Combines IT, hypertext design, content, marketing, art.
- **Young average age**: Less experienced, technology-driven ("tech freaks").
- **Community development**: Open source collaboration, unwritten cooperation rules.

4.2 Technical Infrastructure

- **Inhomogeneity**:
    - Dependence on **external components** (server & browser).
    - No control over user's browser/version/preferences.
- **Immaturity**:
    - Components often **buggy or incomplete** due to time-to-market pressure.
    - **Version updates** can break environments, cause knowledge loss.

4.3 Development Process

- **Flexibility**: Cannot follow rigid plans; must adapt to changes.
- **Parallelism**:
    - **Component-based parallel development** (e.g., authentication, search).
    - **Phases overlap** (design, implementation, testing for different versions).

4.4 Integration

- **Internal integration**: With existing legacy systems (e.g., product catalogs).
- **External integration**: With other Web applications/services.
    - **Challenges**: Many autonomous, heterogeneous, evolving sources.
    - **Web services**: Reusable components with defined interfaces; issues include side effects, QoS.

5. Evolution (Cross-Cutting Dimension)

- **Governs product, usage, and development**.
- **Drivers**:
    - **Continuous change** in requirements, technologies, conditions.
    - **Competitive pressure** – time-to-market critical.
    - **Fast pace** – "either fast or irrelevant" (Tsichritzis).
- **Implications**:
    - **Permanent maintenance** – update cycles of days/weeks.
    - **Need for "lean" Software Engineering processes** with focus on:
        - Requirements analysis
        - Operation & maintenance

7. Why Web Projects Fail – Key Reasons

1. Not meeting functionality/user needs
2. Poor performance
3. Security breaches
4. Poor maintainability
5. Poor scalability & usability
6. Use of immature technology
7. Schedule & cost overruns
8. Poor project management
9. Inability to handle complexity
10. Stakeholder politics
11. Commercial pressures
12. Poor reporting & monitoring